Recommendation report (PDF)

ForwardDeployedEngineerassignment · prepared12October2026

The short answer

I collected 360 real reviews from the MoveInSync app's Google Play page and read them for patterns. The rating is low for a simple reason: the app lets people down at the two moments that matter most in an employee's day. One is when they want to know where their cab is. The other is when they try to book a ride.

- **Live tracking and ETA** is the most talked-about problem: 115 of the 360 reviews. Employees say the cab is late and the tracking is often not available.
- **Booking** comes next with 99 reviews. Three out of four of them are unhappy (75 of 99), the worst share among the big topics.
- **Everything else** (login, crashes, notifications, support, pickup) is smaller. Each still produces one-star reviews, and some of these are probably cheap to fix.

My advice is to spend the first few days proving what is actually going wrong with tracking and booking, using the app's own logs, and then fix what we confirm. At the same time, set up a weekly rating check so that "how are our ratings?" always has the same quick, honest answer. The rest of this report gives the evidence, my reasoning and a plan a product and engineering team could start on tomorrow.

Measure	Value
Reviews collected	360, dated 3 Oct 2018 to 18 Sep
Average rating of these reviews	2026 2.19 out of 5
Reviews giving 1 or 2 stars	239 (66%)
Reviews giving 1 star	205 (57%)
Reviews giving 5 stars	63 (17.5%)

One caution on the 2.19. It is the average of the reviews I collected, so it may not match the rating shown on the store listing. I would always show the two side by side.

What I did, and where the data comes from

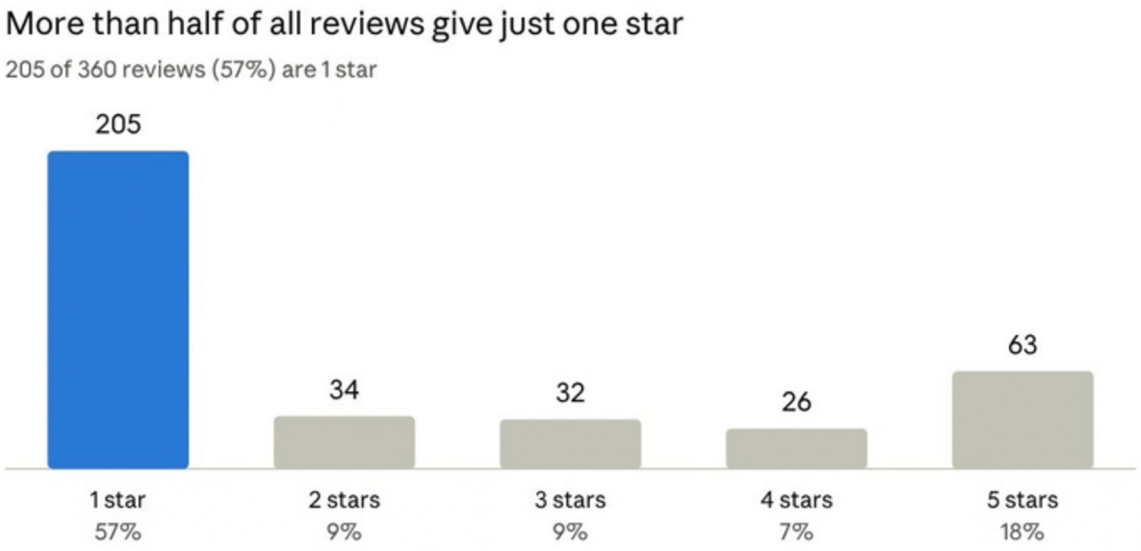
The brief said no data would be prepared for me, so I went and got it. I ran a JavaScript script in the browser's developer console on the public Google Play listing, which pulled each review's rating, date, helpful count and text into a file. That gave me 360 reviews, the oldest from October 2018 and the newest from September 2026.

Before reading anything, I checked the file. Every review has a valid rating from 1 to 5 and a readable date, none are empty, and none are duplicates. I kept every row.

Then I read the reviews and tagged each one with the main problem it talks about. The tagging uses plain keyword rules in a Python script, so I can repeat it next week with exactly the same logic. A review that mentions two problems is counted once, under the main one, so nothing is double counted. These tags are good for deciding where to look first. A person should still read the reviews behind each tag before anything gets built. What I did not have is MoveInSync's own data: app logs, booking records, support tickets. That matters later in this report, because reviews tell us what employees feel, not why it happens.

How people rate the app

People who write reviews tend to be either annoyed or delighted, and here the annoyed clearly win. There are more than three times as many 1-star reviews as 5-star ones (205 against 63). Only 92 reviews sit in the middle, at 2, 3 or 4 stars.



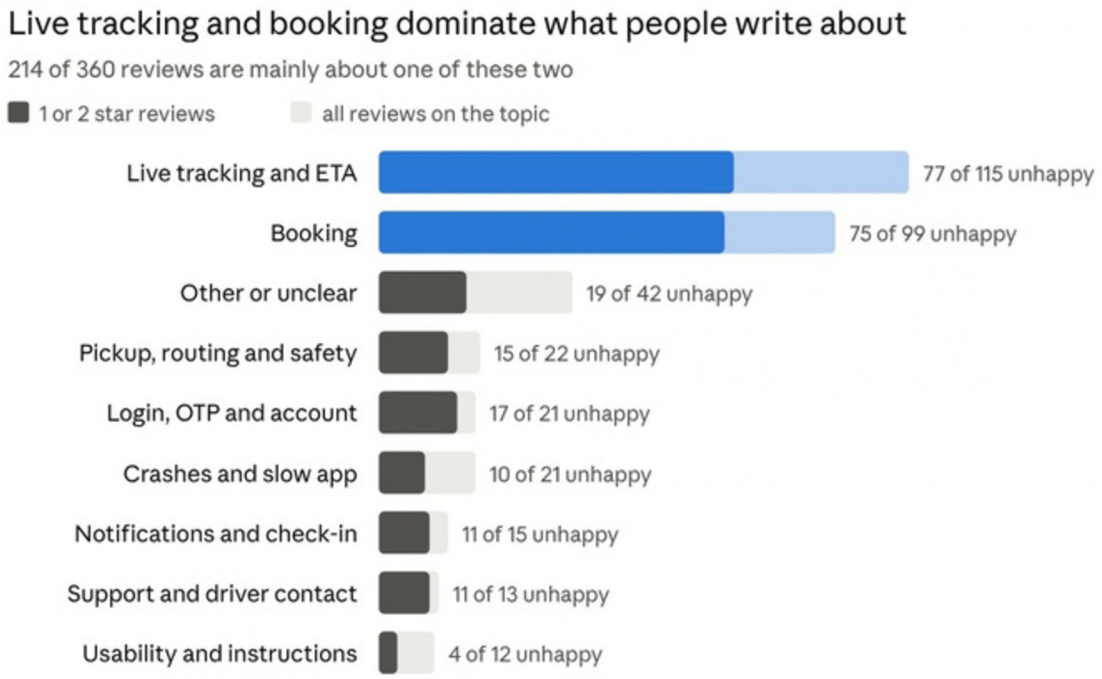
Google Play reviews collected with a console script · 360 reviews, Oct 2018 to Sep 2026

That shape tells us something useful about the fix. Nobody needs to win over a lukewarm middle. The rating moves when we remove the specific problems that push people straight to 1 star. The 63 five-star reviews also show that for some employees the app does

what it should, so this is a reliability problem more than a product that misses the point.

What people are writing about

Two topics stand out from everything else. Live tracking and ETA shows up in 115 reviews and booking in 99. Together that is 214 of 360, about 59%.



Same 360 reviews · each tagged with the main problem it describes

Live tracking and ETA

This is the feature people care about most, and not only in anger. Of the 115 reviews, 77 are unhappy, but 25 give 4 or 5 stars. When tracking works, people say so. When it doesn't, they are standing at a gate wondering whether the cab is coming. One reviewer put it in a single line: "cab are always late and track is also not available many time."

There are really two complaints inside that sentence. The cab is late, which is an operations problem. And the tracking is not available, which is an app problem. The app cannot make a cab arrive sooner, but it can show an honest, fresh location and a believable ETA. A wrong or missing ETA is worse than no ETA, because the employee plans their whole morning around it.

B ooking

Self-booking is the main way trips get created, so booking is the front door of the product. Of 99 booking reviews, 75 are unhappy. That is 76%, the highest unhappy share among the two big topics (tracking is 67%). One reviewer wrote: "loads info very slow, to do the booking."

From reviews alone I cannot tell whether bookings really fail or whether they only feel uncertain, with slow screens and no clear confirmation. Both are bad, but they have different fixes. That is why the first job is to follow a booking all the way from the tap to the confirmed trip.

The smaller topics

- **Login and OTP (21 reviews, 17 unhappy).** People cannot get in, and without login nothing else works. "Wastage of time. I bet if you are able login at once."
- **Crashes and slow app (21 reviews, 10 unhappy).** The most mixed group. One person describes the app "showing not installed" when they open it.
- **Notifications and check-in (15 reviews, 11 unhappy).** At least some people get too many alerts, not too few: "So many Notifications for just one travel." Check-in and no-show rules also sit in this group.
- **Support and driver contact (13 reviews, 11 unhappy).** The count is small and the mood is the worst of any group, at 85% unhappy. These are people who already had a problem and then could not get help.
- **Pickup, routing and safety (22 reviews, 15 unhappy).** I would treat every safety-related review as urgent whatever the count.
- **Usability (12 reviews, 4 unhappy) and other or unclear (42 reviews, 19 unhappy).** Mostly positive or too vague to act on.

Why I think this is happening

Reviews show the symptoms. They cannot show the cause. What follows are my best guesses, ordered by how strongly the reviews point to them, and each comes with a way to prove or kill it. I would rather be wrong quickly in a test than be wrong after a release.

Topic	What the employee lives through	My best guess	How I would check
Live tracking and ETA	Cannot tell when or where to board, and stops trusting the ETA	Old location data, slow refresh, ETA recalculation, network delay, or a screen that hides how fresh the data is	Compare the time on the phone, on the server and on screen; log how old a location is and how often the ETA jumps
Booking	Thinks a ride is booked when it is not, or cannot finish booking	Unclear buttons and status, extra steps, a backend failure, or an outdated roster or address	Track every booking step: roster, select, submit, confirmed by the server
Login and OTP	Locked out of trip details and booking	OTP or session errors, profile or address checks, poor handling of server replies	Track login success, OTP errors and session timeouts
Crashes and slow app	App feels slow or unavailable at commute time	Crashes, slow start, poor handling of network errors, slow server responses	Crash rate, app start time, API speed and error rates
Notifications and check-in	Missed check-in or a surprise no-show	Late or missing alerts, an unclear check-in deadline, rules that do not match when the vehicle arrives	Compare when alerts were sent and opened with when the vehicle arrived

One thing I want to be clear about: I am not claiming the app is broken in every case. A late cab is not an app bug. Part of the job is separating what the app can fix from what operations has to fix, and being honest with employees about the difference.

What I would do first

Week one: find the real cause behind tracking and booking

For **tracking**, I would pull the app and server logs for a few routes and compare three timestamps for each location update: when the vehicle sent it, when the server received it, and when the employee's screen showed it. If the gap is big, that is the bug. I would also log how often the ETA changes and by how much. And I would add a small "last updated"

label with a clear stale state, so the app tells the truth when data is old instead of showing an old dot with confidence. I would not simply refresh faster. That costs battery and server load and may not touch the real cause.

For **booking**, I would book rides myself with test accounts, from the roster to the confirmed trip, and write down every screen where a user could believe they are done when they are not. Then I would measure how many people drop out at each step. The fix is likely about honesty of state: pending, failed and confirmed must look obviously different, and the app should never say "success" before the server has confirmed.

Next: the quicker wins

Login and OTP errors should say what went wrong and what to do next, not fail silently. Notifications should be reviewed against check-in and no-show rules, including phone permission settings. The helpdesk and driver-contact flow should tell people clearly what to do when contact is restricted, and any safety message should reach a human fast.

Priority	What	Who	Effort	How we will know it worked
P0, start now	Reproduce stale location and ETA problems; add a last-updated time and a stale state	Android, Backend, QA	Medium	Location freshness and ETA accuracy are tracked; the issue is reproduced or ruled out
P0, start now	Trace booking from tap to server-confirmed trip; make pending, failed and confirmed unmistakable	Product, Android, Backend	Medium	Every booking step and failure reason is measured; the new states pass a usability test
P1, next	Fix login, OTP, session and profile failure paths with clear errors and a way to recover	Android, Backend, QA	Medium	Login and OTP success rate and error mix are tracked
P1, next	Match notifications to check-in and no-show rules and to vehicle arrival	Android, Transport Ops, Product	Medium	Alert delay and missed check-ins are tracked
P1, next	Review helpdesk, driver contact and safety escalation	Product, Support and Ops	Low to medium	Contact success and ticket resolution time are tracked

Priority	What	Who	Effort	How we will know it worked
P2, ongoing	Weekly review digest with alerts, tied to releases and incidents	Product Analytics, Support	Low to medium	The digest goes out on time and new themes are flagged early

I am ranking by how many users a problem touches and how serious it is, against how much work it takes. I am deliberately not promising a number of rating points. Nobody can honestly forecast that from reviews alone, and a fix that works will show up in the weekly numbers anyway.

The first eight weeks

The timings below are my estimates. They depend on release capacity and on how quickly the logs turn out to be usable.

1. **Days 1 to 3.** Read a fresh batch of complaints, reproduce the tracking and booking problems, find out which crash, booking and login data already exists, and open tickets with clear steps to repeat each problem. This needs Android builds, backend logs, analytics access and QA test accounts. Each P0 item ends with either reproduction steps or a plain "could not reproduce".
2. **Weeks 1 to 3.** Fix the problems we have confirmed. Make booking states clear and add proper error recovery. Agree the event list the app should log. Review notifications and support escalation. The exit test is simple: regression tests pass and the new events show up in the data.
3. **Weeks 4 to 8.** Build dashboards that split results by app version, phone model and network. Add safeguards and alerts for stale data where the diagnosis supports them. This needs stable event definitions and a privacy review first.
4. **Every week after that.** Collect a fresh batch of reviews, tag them, compare against the last few weeks, and connect the themes to releases and support tickets.

Answering "how are our ratings?" every week

This question will keep coming, so the answer should take five minutes to produce and look the same every time. I would send one page, with these things in the same order each week.

What goes on the page	Why it is there
The rating shown on Google Play, and the date it was read	It is the number the public sees
Average of the freshly collected reviews, how many there are, and the 1 to 5 star split	Shows movement that the single listing number hides
Number of 1 and 2 star reviews, and the top three topics	Tells us where the pain is this week
New topics versus ones we have seen before	Separates a fresh regression from a long-running problem
The app release or incident dates for the week	Lets us link a change in rating to a cause
Proposed action, owner and status	Makes sure someone acts on it

I would run the same console script each time, and note the date, region and language, so one week can be compared fairly with the next.

Suggested alert rules. These are my starting point, to be adjusted once we have a few weeks of data:

- Send any credible safety concern to Support or Transport Operations immediately.
- Flag a problem area when its negative reviews jump 50% above its usual eight-week level, and there are at least 5 of them.
- Flag a drop of 0.3 in the average rating compared with the last four weeks.
- Review and adjust these rules after 8 to 12 weeks.

Product Analytics owns the page, Support triages the reviews, and Product and Engineering own the fixes.

What I would need from MoveInSync

To turn my guesses into facts, I would ask for these, roughly in this order:

- Location update logs with phone, server and display times, for a few routes
- Booking events from roster to confirmed trip, including failures and their reasons
- Crash and slow-start data by app version and phone model
- Login and OTP success and error logs
- Notification send, delivery and open logs, next to vehicle arrival times
- Support tickets and driver-contact records, with how long each took to resolve

- App release dates, so I can see whether a bad month of reviews follows a release
- The rating currently shown on the Play listing, and the exact steps used to collect the reviews, so the first weekly page has a proper baseline

Where this could be wrong

I would rather say this myself than have someone find it. The reviews are real, but they are what people chose to write, and unhappy people write more often than happy ones. My topic tags come from keyword rules, so a few reviews will sit in the wrong group, and one person should read through them before anything is built. I have not split the reviews by year or app version, so I cannot yet say whether things are getting better or worse. And without MoveInSync's internal data I cannot prove any cause, only point to where to look. None of that changes the first move. Tracking and booking are the clearest signals in the data, they sit at the two most important moments for an employee, and checking them is cheap.

How I did this, and the tools I used

I collected the reviews from the Google Play listing with a JavaScript script run in the browser's developer console. I cleaned and tagged them with Python (pandas, numpy and matplotlib), and used Claude, an AI assistant, to help read the results, test my reasoning. The analysis script and the tagged file can be re-run, so next week's numbers will be produced the same way as these.